

Contents

Sistema de Scraping e Scoring de Apostas - BetinAsia	1
Documento Técnico de Arquitetura	1
1. Visão Geral do Sistema	2
1.1 Objetivo	2
1.2 Arquitetura Geral	2
1.3 Fluxo de Dados	2
2. Componentes do Sistema	3
2.1 SCRAPING (Playwright)	3
2.2 QUEUE/SCHEDULER	5
2.3 CACHE (Redis)	6
2.4 STORAGE (PostgreSQL)	7
2.5 SCORING ENGINE	9
2.6 MONITORING	11
3. Fontes de Dados Externas	11
3.1 Closing Line	11
3.2 Resultados de Jogos	12
3.3 Cálculo de Resultado Asian Handicap	13
4. Resumo de Custos	13
4.1 Opção Mínima (Self-hosted)	13
4.2 Opção Intermediária (Cloud services)	14
5. Cronograma de Implementação	14
6. Complexidade por Componente	15
7. Stack Tecnológico Recomendado	15
8. Próximos Passos	16

Sistema de Scraping e Scoring de Apostas - BetinAsia

Documento Técnico de Arquitetura

Versão: 1.0

Data: Janeiro 2026

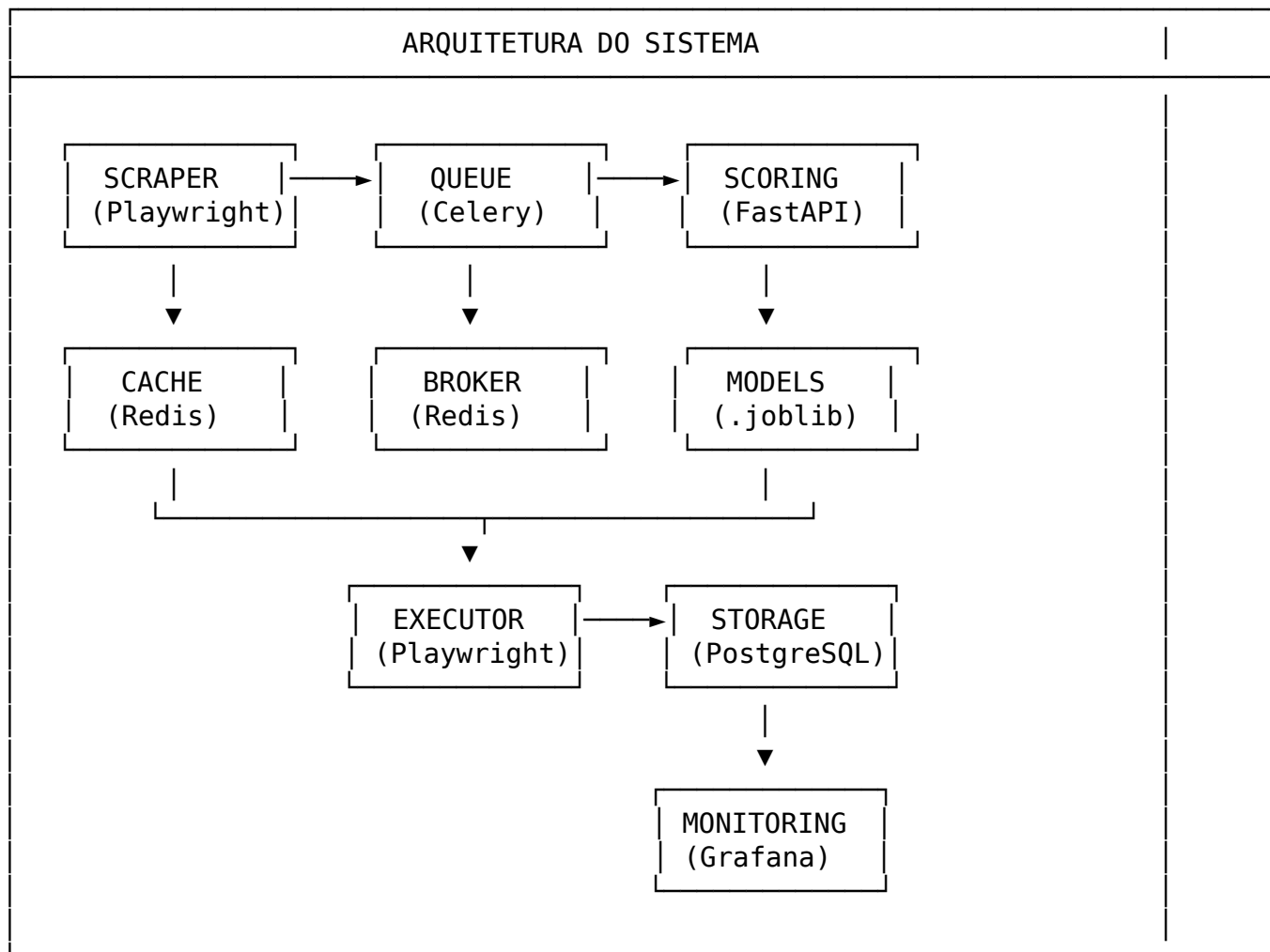
Projeto: Operação Profissional de Apostas Esportivas

1. Visão Geral do Sistema

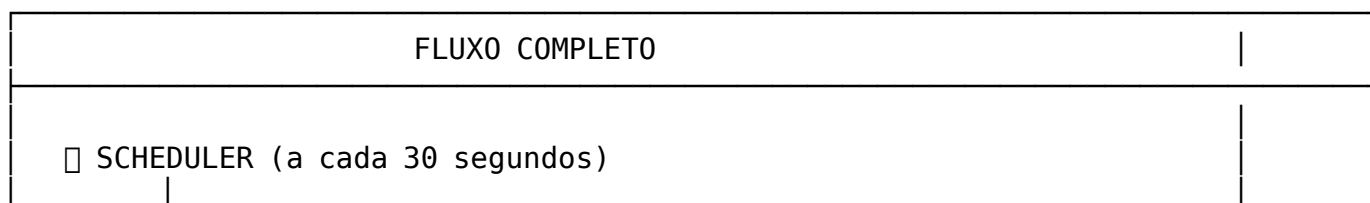
1.1 Objetivo

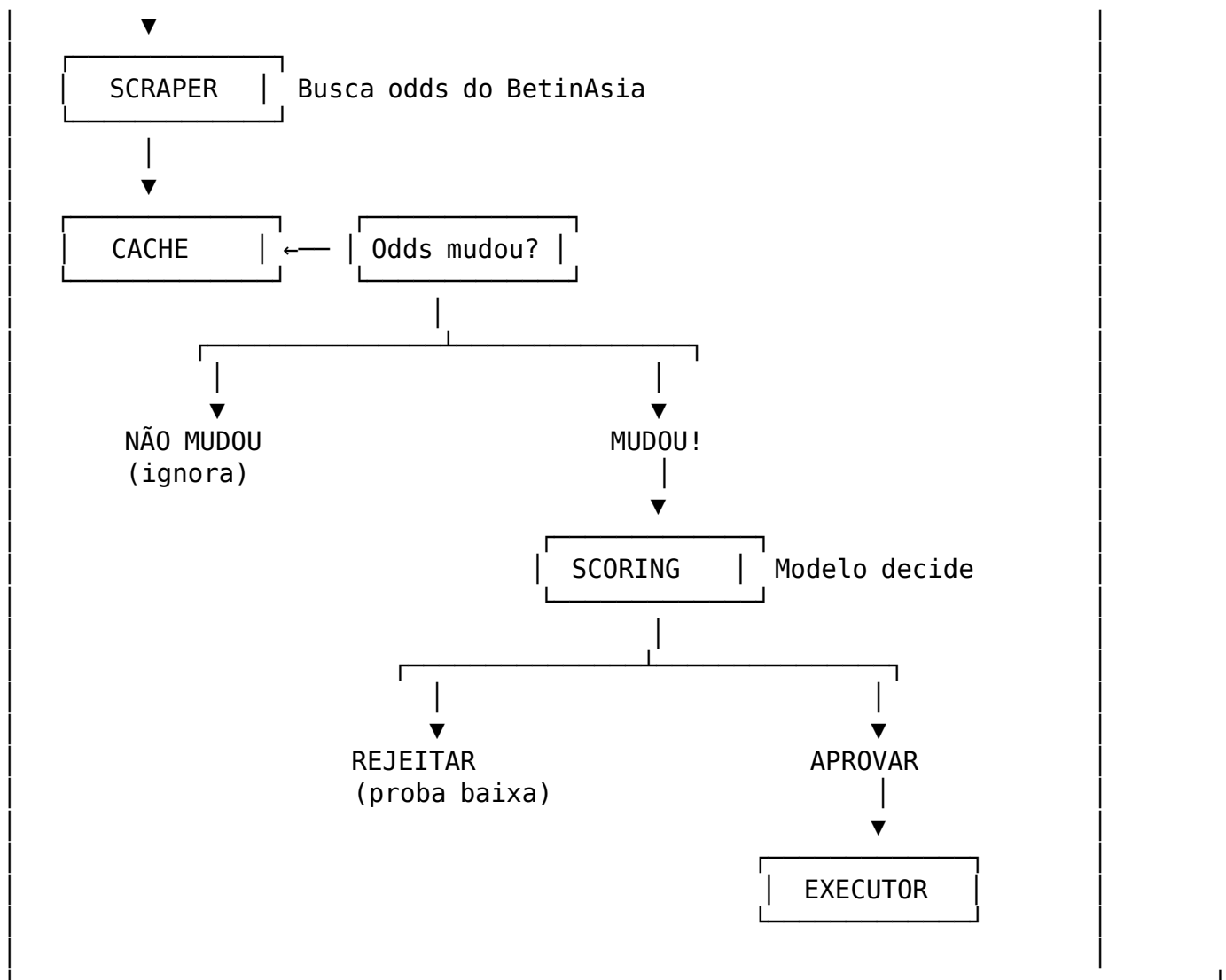
Construir um sistema automatizado para: - Fazer scraping de odds do BetinAsia - Processar oportunidades através de modelo estatístico - Executar apostas automaticamente quando aprovadas

1.2 Arquitetura Geral



1.3 Fluxo de Dados





2. Componentes do Sistema

2.1 SCRAPING (Playwright)

O que é

Playwright é uma biblioteca de automação de browser desenvolvida pela Microsoft para fazer web scraping de sites com JavaScript dinâmico.

Características

Aspecto	Detalhe
Tecnologia	Playwright (Python)
Complexidade	8/10
Custo	Gratuito (open source)
Tempo de desenvolvimento	2-4 semanas

Desafios Técnicos

- Entender estrutura HTML/JS do BetinAsia
- Lidar com carregamento dinâmico (AJAX/WebSocket)
- Manter sessão de login ativa
- Detectar e contornar anti-bot (se existir)
- Tratar mudanças no layout do site

Estrutura de Dados

```
@dataclass
class BookmakerOdds:
    bookmaker: str
    home_odds: float
    away_odds: float
    timestamp: datetime
```

```
@dataclass
class AHLLine:
    line: str # Ex: "+0.5", "-0.75"
    bookmaker_odds: dict[str, BookmakerOdds]
```

```
@dataclass
class MatchData:
    match_id: str
    league: str
    home_team: str
    away_team: str
    kickoff_time: datetime
    ah_lines: dict[str, AHLLine]
```

Instalação

```
pip install playwright
playwright install chromium
```

2.2 QUEUE/SCHEDULER

Conceito Simplificado

Scheduler = “Despertador” que dispara tarefas em horários programados.

Queue = “Fila de espera” onde tarefas aguardam para serem executadas.

Funcionamento

SCHEDULER (Agenda)
□ A cada 30 segundos: → Verificar Premier League, La Liga, Serie A □ A cada 2 minutos: → Verificar Championship, Eredivisie □ A cada 5 minutos: → Verificar ligas menores □ Todos os dias às 23:59: → Gerar relatório do dia

Opções Disponíveis

Opção	Complexidade	Custo	Quando usar
APScheduler	3/10	Gratuito	Início/projetos simples
Celery + Redis	6/10	Gratuito (local) ou R\$ 30-50/mês	Produção/escala

Configuração de Ligas por Prioridade

```
priority_leagues:
  tier_1: # Alta frequência (30-60s)
    - "England Premier League"
    - "Germany Bundesliga"
    - "Spain La Liga"
    - "Italy Serie A"
    - "France Ligue 1"

  tier_2: # Frequência média (2-3min)
```

- "England Championship"
- "Germany 2. Bundesliga"
- "Netherlands Eredivisie"
- "Portugal Primeira Liga"

`tier_3: # Frequência baixa (5-10min)`

- "Belgium Pro League"
- "Brazil Serie A"
- "Turkey Super Lig"

2.3 CACHE (Redis)

Conceito Simplificado

Cache é como um "Post-it" que guarda informações recentes para não precisar buscar novamente.

Uso no Sistema

CACHE NO SISTEMA DE APOSTAS
PROBLEMA SEM CACHE: 12:00:00 - Scrape: odds = 1.95 12:00:30 - Scrape: odds = 1.95 → Processou de novo (desnecessário) 12:01:00 - Scrape: odds = 1.95 → Processou de novo (desnecessário) SOLUÇÃO COM CACHE: 12:00:00 - Scrape: odds = 1.95 → Salva no cache 12:00:30 - Scrape: odds = 1.95 → Igual ao cache → IGNORA 12:01:00 - Scrape: odds = 1.98 → Diferente! → PROCESSA

Estrutura do Cache

ESTRUTURA DO CACHE		
Chave	Valor	Expira em
ManCity_Liverpool_+0.5 →	1.98	5 minutos
RealMadrid_Barcelona_-0.75 →	2.05	5 minutos
Bayern_Dortmund_+0 →	1.92	5 minutos

Características

Aspecto	Detalhe
Tecnologia	Redis (ou dicionário Python para início)
Complexidade	3/10
Custo	Gratuito (local) ou R\$ 0-30/mês (cloud)

Usos Principais

Uso	Descrição
Lembrar última odd	Só processa se a odd mudou
Evitar apostas duplicadas	Verifica se já apostou neste jogo
Rate limiting	Controla frequência de requests
Sessão de login	Mantém cookies/tokens

2.4 STORAGE (PostgreSQL)

Características

Aspecto	Detalhe
Tecnologia	PostgreSQL
Complexidade	4/10
Custo	Gratuito (local) ou R\$ 0-25/mês (Supabase)

Schema do Banco

-- Tabela de partidas

```
CREATE TABLE matches (  
  id SERIAL PRIMARY KEY,  
  external_id VARCHAR(100) UNIQUE NOT NULL,  
  league VARCHAR(200) NOT NULL,  
  home_team VARCHAR(200) NOT NULL,  
  away_team VARCHAR(200) NOT NULL,  
  kickoff_time TIMESTAMPTZ NOT NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Tabela de odds (histórico)

```

CREATE TABLE odds_history (
    id SERIAL PRIMARY KEY,
    match_id INTEGER REFERENCES matches(id),
    ah_line VARCHAR(20) NOT NULL,
    bookmaker VARCHAR(50) NOT NULL,
    home_odds DECIMAL(5,3) NOT NULL,
    away_odds DECIMAL(5,3) NOT NULL,
    scraped_at TIMESTAMPTZ DEFAULT NOW()
);

-- Tabela de oportunidades
CREATE TABLE opportunities (
    id SERIAL PRIMARY KEY,
    match_id INTEGER REFERENCES matches(id),
    ah_line VARCHAR(20) NOT NULL,

    -- Odds de detecção
    detection_odds DECIMAL(5,3),
    detection_time TIMESTAMPTZ,

    -- Closing line
    closing_odds DECIMAL(5,3),
    closing_time TIMESTAMPTZ,

    -- CLV
    clv DECIMAL(6,4),
    clv_positive BOOLEAN,

    -- Scoring
    proba DECIMAL(6,5),
    cutoff DECIMAL(4,3),
    decision BOOLEAN,

    -- Resultado
    home_score INTEGER,
    away_score INTEGER,
    bet_result VARCHAR(20),
    profit_loss DECIMAL(10,4),

    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Tabela de apostas executadas
CREATE TABLE bets (
    id SERIAL PRIMARY KEY,
    opportunity_id INTEGER REFERENCES opportunities(id),
    match_id INTEGER REFERENCES matches(id),
    ah_line VARCHAR(20) NOT NULL,

```



```

side VARCHAR(10) NOT NULL,
bookmaker VARCHAR(50) NOT NULL,
expected_odds DECIMAL(5,3) NOT NULL,
actual_odds DECIMAL(5,3),
stake DECIMAL(10,2) NOT NULL,
status VARCHAR(20) NOT NULL,
confirmation_id VARCHAR(100),
result VARCHAR(20),
profit_loss DECIMAL(10,2),
created_at TIMESTAMPTZ DEFAULT NOW(),
executed_at TIMESTAMPTZ,
settled_at TIMESTAMPTZ,

UNIQUE (match_id, ah_line, side, bookmaker)
);

```

2.5 SCORING ENGINE

Nova Abordagem (Sem RebelBetting)

Como o sistema opera diretamente no BetinAsia, não teremos a feature Dif Odds RB & BIA. Necessário:

1. Coletar novos dados (2-4 semanas)
2. Criar novas features
3. Treinar novo modelo

Novas Features Propostas

Features Numéricas:

Feature	Descrição
num_bookmakers	Número de casas oferecendo a linha
dif_pct_best_second	Diferença % entre melhor e segunda melhor odd
dif_pct_best_median	Diferença % entre melhor odd e mediana
dif_vs_pinnacle	Diferença vs Pinnacle (benchmark)
home_away_spread	Spread entre odds home/away
minutes_to_kickoff	Minutos até o início do jogo
odds_volatility	Desvio padrão das últimas odds
odds_trend	Tendência de movimento

Features Categóricas:

Feature	Descrição
weekday	Dia da semana (0-6)
turno	Período do dia (manhã/tarde/noite)
ah_line	Linha de handicap (+0.5, -0.75, etc.)
best_bookmaker	Casa com melhor odd
league	Liga/competição

Processo de Treinamento

PROCESSO DE TREINAMENTO
FASE 1: COLETA (2-4 semanas) • Scraping contínuo do BetinAsia • Registrar todas as oportunidades detectadas • NÃO apostar, apenas coletar • Obter closing lines (scrape final antes do jogo) • Obter resultados das partidas FASE 2: PREPARAÇÃO DOS DADOS • Calcular CLV: $(\text{detection_odds} - \text{closing_odds}) / \text{closing_odds}$ • Criar target: $\text{clv_positive} = \text{CLV} > 0$ • Limpar dados (outliers, missing values) FASE 3: TREINAMENTO • Split temporal (80% treino, 20% teste) • Validação cruzada com TimeSeriesSplit • Modelo: Regressão Logística com regularização • Métricas: AUC-ROC, Precision, Recall FASE 4: VALIDAÇÃO • Avaliar no conjunto de teste • Analisar por cutoff (0.50, 0.55, 0.60, 0.65) • Definir cutoff ótimo (equilíbrio volume vs qualidade)

Características

Aspecto	Detalhe
Tecnologia	scikit-learn (Logistic Regression)
Complexidade	7/10
Tempo de coleta	2-4 semanas
Tempo de treinamento	2-3 dias
Custo	Gratuito

2.6 MONITORING

Opções Disponíveis

Opção	Complexidade	Custo	Descrição
Logs JSONL	2/10	Gratuito	Simples, bom para início
Prometheus + Grafana	5/10	Gratuito (local)	Dashboards visuais

Métricas Importantes

Scraping: - Total de requests - Taxa de sucesso/erro - Duração média por liga - Partidas encontradas

Scoring: - Total de scorings - Distribuição de probabilidades - Taxa de aprovação - Duração da inferência

Execução: - Total de apostas - Status (placed, rejected, error) - Distribuição de stakes

P&L: - P&L diário/acumulado - ROI - Win rate - Drawdown

3. Fontes de Dados Externas

3.1 Closing Line

Estratégia Recomendada

Usar o próprio BetinAsia: - Agendar scrape final 5-10 minutos antes do kickoff - Salvar como closing_odds - Calcular CLV comparando com detection_odds

COLETA DE CLOSING LINE	
14:00	Detecta oportunidade: odds = 1.95

	→ Agenda scrape de closing para 15:50
15:50	Scrape final: odds = 1.88
	→ Salva: closing_odds = 1.88
	→ Calcula: CLV = (1.95 - 1.88) / 1.88 = +3.7%
16:00	Jogo começa

3.2 Resultados de Jogos

APIs Disponíveis

API	Custo	Cadastro	Dados
API-Football	Grátis (100 req/dia)	Email + senha, 2 min	Completo
Football-Data.org	Grátis (10 req/min)	Email + senha, 2 min	Ligas principais
The Odds API	Grátis (500 req/mês)	Email + senha, 2 min	Odds + Resultados

Processo de Cadastro

1. Acessar o site da API
2. Criar conta (email + senha)
3. Confirmar email
4. Receber API Key instantaneamente
5. Usar no código

Não precisa: aprovação manual, documentos, esperar dias.

Exemplo de Uso

```
import requests
```

```
API_KEY = "sua_api_key_aqui"
```

```
response = requests.get(
    "https://v3.football.api-sports.io/fixtures",
    headers={"X-Auth-Token": API_KEY},
    params={
        "league": 39, # Premier League
        "season": 2025,
        "date": "2026-01-28"
    }
)
```

```
jogos = response.json()
for jogo in jogos["response"]:
    print(f"{jogo['teams']['home']['name']} "
          f"{jogo['goals']['home']} x "
          f"{jogo['goals']['away']} "
          f"{jogo['teams']['away']['name']}")
```

3.3 Cálculo de Resultado Asian Handicap

```
def calcular_resultado_ah(home_score: int, away_score: int,
                          ah_line: str, side: str) -> tuple[str, float]:
    """
    Calcula resultado de aposta Asian Handicap.

    Args:
        home_score: Gols do time da casa
        away_score: Gols do visitante
        ah_line: Linha de handicap (ex: "+0.5", "-1.25")
        side: "home" ou "away"

    Returns:
        (resultado, lucro_por_unidade)
    """
    line = float(ah_line)

    if side == "home":
        adjusted_diff = (home_score + line) - away_score
    else:
        adjusted_diff = (away_score + line) - home_score

    if adjusted_diff > 0.5:
        return ("win", 1.0)
    elif adjusted_diff > 0:
        return ("half_win", 0.5)
    elif adjusted_diff == 0:
        return ("push", 0.0)
    elif adjusted_diff > -0.5:
        return ("half_loss", -0.5)
    else:
        return ("loss", -1.0)
```

4. Resumo de Custos

4.1 Opção Mínima (Self-hosted)

Componente	Custo
VPS Linux (4GB RAM)	R\$ 50-100/mês
Playwright	Gratuito
APScheduler	Gratuito
PostgreSQL (local)	Gratuito
Redis (local)	Gratuito
Logs JSONL	Gratuito
TOTAL	R\$ 50-100/mês

4.2 Opção Intermediária (Cloud services)

Componente	Custo
VPS Linux (8GB RAM)	R\$ 150-250/mês
Playwright	Gratuito
Celery	Gratuito
Redis Cloud	R\$ 30-50/mês
Supabase PostgreSQL	Gratuito ou R\$ 125/mês
Grafana Cloud	Gratuito ou R\$ 75/mês
TOTAL	R\$ 180-500/mês

5. Cronograma de Implementação

CRONOGRAMA SUGERIDO
FASE 1: FUNDAÇÃO (Semana 1-2) — Configurar VPS e ambiente — Instalar PostgreSQL + Redis — Criar estrutura de projeto — POC de login no BetinAsia FASE 2: SCRAPING (Semana 2-4) — Desenvolver scraper completo — Testar com 2-3 ligas — Implementar cache de odds — Configurar scheduler básico FASE 3: COLETA DE DADOS (Semana 4-8) — Rodar sistema em modo coleta — NÃO apostar, apenas coletar features — Obter resultados das partidas

└─ Calcular CLV

FASE 4: MODELO (Semana 8-9)

- └─ Treinar modelo com dados coletados
- └─ Validar performance (AUC, calibração)
- └─ Definir cutoff ótimo
- └─ Integrar engine de scoring

FASE 5: EXECUÇÃO (Semana 9-10)

- └─ Implementar executor de apostas
- └─ Testar com stakes pequenos
- └─ Configurar alertas (Telegram)
- └─ Setup de monitoring

FASE 6: PRODUÇÃO (Semana 10+)

- └─ Monitorar performance real
- └─ Ajustar cutoff se necessário
- └─ Expandir para mais ligas
- └─ Retreinar modelo periodicamente

TOTAL ESTIMADO: 6-10 semanas

6. Complexidade por Componente

Componente	Complexidade	Tempo Setup	Dependências
Scraping (Playwright)	8/10	2-4 semanas	Entender HTML do site
Queue/Scheduler	3-6/10	1-2 dias	Redis (opcional)
Storage (PostgreSQL)	4/10	1 dia	PostgreSQL
Cache (Redis)	3/10	0.5 dia	Redis
Scoring (treino)	7/10	2-4 semanas	Dados coletados
Scoring (engine)	4/10	1 dia	Modelo treinado
Execução	7/10	1-2 semanas	Scraper funcionando
Monitoring	2-5/10	1-2 dias	Prometheus/Grafana

7. Stack Tecnológico Recomendado

Componente	Tecnologia	Justificativa
Scraping Queue/Scheduler	Playwright (Python) APScheduler (início) ou Celery	Headless, async, bom com JS Simplicidade vs Escala
Storage	PostgreSQL	Robusto, JSON nativo, gratuito
Cache	Redis ou dict Python	Performance, simplicidade
Scoring	scikit-learn	Maduro, simples, eficaz
Monitoring	Logs JSONL (início) ou Grafana	Simplicidade vs Visualização
Alertas	Telegram Bot	Gratuito, fácil de usar

8. Próximos Passos

1. **Criar conta em VPS** (DigitalOcean, Vultr, Hetzner)
2. **Fazer engenharia reversa do BetinAsia** (F12 → Network)
3. **Desenvolver POC de scraping** com 1 liga
4. **Iniciar coleta de dados** para treinamento do modelo
5. **Cadastrar em API de resultados** (API-Football ou Football-Data)

Documento gerado em Janeiro 2026